

Asymptotic

Incremental Technical Audit

Repeated proof search, optimization-sensitive checks,
and comment-blind mathematical cross-references

Repository VladimirReshetnikov/Asymptotic

Reviewed commit 8cee870994f506b501bae3ea6bd4a3a7edb895c1

Target interpreters Wolfram Language 15 and Mathics3

Review date 10 September 2026

Findings and evidence

Three retained findings: one Mathics-path performance defect and two documentation-validation defects. Seven focused pinned-package controls agree with native or closed-form oracles in Wolfram 15.0.0. Retrieved proof helpers reproduce the predicted small-depth counts on the native host. Nineteen bundle tests pass; 56 documentation-fixture observations and five two-pass LaTeX experiments are supplied.

Mathics itself and the full repository test suite were not executed. The large proof-search comparison is a source-derived, independently checked model, not a Mathics timing result. Candidate repairs are separate from the reviewed source.

Abstract

This report examines a pinned revision of the Asymptotic repository while excluding the substantial existing review backlog from its new finding count. The most significant additional implementation mechanism identified is duplicated work in the mutually recursive Mathics realness and sign provers. A nested expression containing only 37 leaves induces 335,521 helper-body evaluations in a source-faithful model at the existing depth limit of 24. Query-local memoization reduces that model to 200 distinct body evaluations without changing its inconclusive answer. An exact recurrence explains both the initial exponential growth and the fixed-depth plateau; executions of the retrieved helpers in Wolfram 15.0.0 confirm the initial sequence through nesting depth six.

Two independently reproduced defects concern the mathematical documentation gate. Five production checks use Python assertions, so optimized execution can accept deliberately invalid fixtures and still report zero missing references or citations. Separately, regular-expression extraction treats commented-out TeX definitions as active labels and bibliography entries. This masks genuine undefined references even in ordinary Python execution. It also creates false duplicate-label alarms and attempts to read commented-out inputs. Real LaTeX runs distinguish these cases from valid documents.

The report gives concrete witnesses, severity and reachability boundaries, mathematical derivations, candidate Python repairs, portable reproduction specifications, and focused acceptance criteria. It does not claim a newly demonstrated incorrect asymptotic formula, universal Wolfram/Mathics equivalence, or an integrated production fix. Novelty screening used the maintained all-wave registers and indexes, with targeted inspection of nearby findings and the fifth-wave documentation review; it was not a fresh cover-to-cover rereading of every retained article.

Contents

1	Executive assessment	3
2	Snapshot, scope, and non-duplication	3
2.1	A fixed source baseline	3
2.2	How existing findings were excluded	4
3	Execution evidence and its limits	5
3.1	What actually ran	5
3.2	The independently executed lanes	5
4	N01: duplicated Mathics proof search	6
4.1	The exact source mechanism	6
4.2	A compact witness family	6
4.3	An exact call-count recurrence	6
4.4	Measured and modeled results	7
4.5	A repair with a precise equivalence obligation	8
4.6	Focused acceptance criteria	9
5	N02: documentation checks disappear under optimization	9

5.1	Five assertions implement production validity gates	9
5.2	Controlled execution of the actual checker	9
5.3	The candidate fix	10
5.4	Acceptance and remaining scope	10
6	N03: comments alter mathematical cross-reference validity	11
6.1	Source text is not the same as active TeX	11
6.2	Compiler cross-checks	11
6.3	A narrow lexical repair	12
6.4	A useful next development step	12
7	Implementation package and reproducibility	13
7.1	What the supplied code does	13
7.2	Reproducing the local evidence	13
7.3	Testing the candidate in a real checkout	14
8	Prioritized, finding-specific development plan	14
9	Conclusion	14
A	Evidence inventory	15

1 Executive assessment

The new work identified here is narrower than the project’s entire development agenda. The retained findings address a specific proof-search call graph and two specific ways a documentation checker can disagree with its stated validation result. They are not another proposal to add a generic budget, a generic cache, or a generic continuous-integration gate.

ID	Priority	Finding	Strongest evidence
N01	Medium; potentially costly	The Mathics sign/realness recursion repeatedly proves identical subqueries before its depth cap is reached.	Retrieved helper execution on Wolfram; exact recurrence and Python model
N02	Medium	Python optimization removes documentation validity and archive-preservation checks, while success fields remain unconditional.	Verbatim upstream checker under normal and optimized fixture execution
N03	Medium	Commented TeX labels and bibliography entries can conceal unresolved references; commented inputs and labels can also cause false failures.	Checker fixtures and five real two-pass LaTeX experiments

These priorities describe the affected contracts, not a claim that the package currently returns incorrect mathematics on ordinary inputs. N01 is a performance defect in code installed for Mathics. Its source mechanism was exercised on a native host, but its end-to-end cost in Mathics remains unmeasured. N02 and N03 affect development tooling used to validate the supporting article; they do not alter the symbolic kernel’s coefficients.

Recommended first action. Integrate the small documentation repair in an isolated checkout and rerun the real documentation command with its real dependencies. In parallel, add the N01 helper-count witness to the Mathics characterization set before integrating a query-local memoization change. The supplied memoizing model establishes the intended scheduling improvement, not a production replacement for the complete Mathics theorem prover.

2 Snapshot, scope, and non-duplication

2.1 A fixed source baseline

The review pins commit `8cee870994f506b501bae3ea6bd4a3a7edb895c1`. The retrieved commit metadata dates it to 10 September 2026 at 17:10:56 UTC. Source links throughout this article use that immutable revision rather than moving `main`. The generated standalone entry is `AsymptoticAnalysis.wl`; canonical implementation files are under `src/Kernel.[1]`

Inspection concentrated on the following connected areas. The table describes inspected code, not a claim that every file in the repository was read line by line.

Area	Inspected implementation or record
Proof and compatibility	<code>MathicsAssumptions.wl</code> , <code>MathicsSimplification.wl</code> , <code>MathicsCompatibility.wl</code> , <code>MathicsTaylor.wl</code> , <code>MathicsNumerical.wl</code> , and <code>MathicsInverseBranches.wl</code> .
Series algebra	Core sparse and precision-tracked jet definitions; forward powers, logarithms, exponentials, and analytic composition; substantial contiguous portions of <code>SeriesOperations.wl</code> ; <code>GammaInverseOperations.wl</code> .
Special functions	<code>SpecialFunctionIdentities.wl</code> , <code>ParameterizedSpecialFunctions.wl</code> , and <code>DirichletSpecialFunctions.wl</code> .
Review history	The main status register, wave-3 and wave-4 intake/crosswalks and proposals, wave-5 index, retirement records, and targeted nearby review material.
Documentation gate	The complete <code>validation/check_documentation.py</code> , followed by verbatim-source fixture execution and real compiler controls.

This was not a fresh full-suite release qualification, an exhaustive native subsumption test, or a complete audit of the vendored mathematical corpus. Those boundaries matter especially because a large repository can contain sound local implementations while still having untested interactions.

2.2 How existing findings were excluded

The maintained history records 231 attributed entries through the first four waves, with overlaps consolidated rather than counted as separate defects. The fifth-wave index adds reports 37–45 and includes retirement tombstones. The all-wave register and indexes, rather than the presence or absence of an old directory, were used to avoid treating a removed report as newly undiscovered work.[5, 6, 7, 8]

Several tempting candidates were discarded after cross-checking. For example, the zero-order differentiation/cutoff inconsistency is already W3-14. Other inspected paths mapped to existing items rather than surviving as new findings. This article does not restate their repairs or repackage their roadmaps.

The following table records only the closest overlap boundaries for the three retained items. It is a novelty crosswalk, not a second backlog.

New ID	Nearby prior material	Additional mechanism retained here
N01	P06/P08; W3-15; W4-16; R39/R42 affine-classification work	A specific mutually recursive Mathics proof graph, a compact witness family, exact call recurrences, native helper counts, and a depth-sensitive memoization equivalence argument. It is not option-tree traversal, retained frontier key membership, or interval-affinity classification.
N02	V02; W4-09; R39’s receipt-checking item	Optimization removes the actual <code>check_documentation.py</code> predicates. It is not package-load admission, empty-suite detection, or inconsistency inside a submitted receipt.
N03	W3-11; W4-14; R44’s TeX recorder work	Comment text changes mathematical reference validity before compilation. It is not standalone dependency spelling, suite discovery, or retention of recorder inputs across build passes.

The fifth-wave documentation review was checked directly for its finding list and additional proposal boundaries. Its publication, reconciliation, recorder, concurrent-writer, and cache-freshness topics are not repeated here.[9] No matching mechanism for N01–N03 was located in the compared records. This is a bounded statement about the inspected snapshot and records, not a claim to have freshly reread every word of all retained reports.

3 Execution evidence and its limits

3.1 What actually ran

A connected Wolfram kernel reported:

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

The pinned standalone package subsequently loaded successfully using the path returned inside the downloaded `File` object. Initial failed loading/service attempts produced no usable package result and are not attributed to the repository. Seven focused package controls then matched their expected finite expressions. These observations and their exact submitted code are transcribed in `evidence/wolfram-observations.md`.

Control	Matched finite expression	Returned tail
e^x	$1 + x + x^2/2 + x^3/6$	$O(x^4)$
$\sin x$	$x - x^3/6$	$O(x^5)$
$\log(1 + x)$	$x - x^2/2 + x^3/3$	$O(x^4)$
$\sqrt{1 + x}$	$1 + x/2 - x^2/8 + x^3/16$	$O(x^4)$
$(1 - x)^{-1}$	$1 + x + x^2 + x^3$	$O(x^4)$
e^{x+x^2}	$1 + x + 3x^2/2 + 7x^3/6$	$O(x^4)$
Inverse of $x + x^2$	$y - y^2 + 2y^3$, checked against $(\sqrt{1 + 4y} - 1)/2$	Finite-expression control

The forward requests used `"Backend"->"Package"` and cutoff 4. A missing power in the sine expansion legitimately permits a sharper returned tail; a cutoff is not necessarily the exponent of the next nonzero coefficient. These controls establish successful execution of selected code, not proof of every remainder theorem or of every advertised input class.

The actual Mathics proof-helper definitions were also loaded and instrumented in the native kernel. This separate experiment emitted an `Element::shdw` warning because the compatibility context was deliberately added alongside `System`. The warning is preserved in the transcript; the experiment is not described as message-free compatibility testing.

3.2 The independently executed lanes

The local evidence comprises 19 passing Python test methods, 56 documentation-fixture observations, 15 proof-cost model observations, and five LaTeX documents compiled twice each. The documentation fixtures execute the retrieved checker itself. Only its guide/Pandoc, link, encoding, and historical Git providers are replaced by deterministic stubs. Therefore the observations establish the checker's control flow on constructed inputs, not the condition of the live documentation tree.

Two compiler witnesses deliberately retain undefined references or citations. Their `pdflatex` processes exit successfully while the logs contain the expected warnings. These are successful counterexample experiments, not successful reference validation.

Mathics was not executed, and no full original MUnit suite was run. The local-path Wolfram/Mathics reproduction scripts are supplied as specifications for fresh processes; their Mathics runs are unexecuted. No counts from these different lanes are combined into one fictional package acceptance total.

4 N01: duplicated Mathics proof search

4.1 The exact source mechanism

`MathicsAssumptions.wl` contains two mutually recursive bounded procedures: `mathicsRealProof` and `mathicsSignProof`. They receive an expression, retained facts, and a depth budget. Their return values distinguish proved information from an inconclusive `None`. The simplification adapter uses these procedures before or around native Mathics simplification.[2, 3]

The problematic interaction is specific. To establish the realness of a power, the realness prover first verifies the exponent, then asks for signs of the base, and may later ask for realness of the same base. To establish the sign of `Cosh[z]`, the sign prover asks whether `z` is real. When that is not proved, it eventually falls back to asking whether the complete `Cosh[z]` is real. The latter query asks for realness of `z` again, at another depth. Nested powers and hyperbolic cosines multiply these repeated subqueries.

The depth counter prevents this particular recursion from being infinite, but does not prevent extensive duplicated work. In the relevant pure fragment, every recursive edge decreases depth. The implementation nevertheless traverses a tree of calls instead of sharing the repeated nodes of that finite dependency graph.

4.2 A compact witness family

Let a be an unassigned symbolic parameter, with no realness or sign facts, and define

$$E_0 = a, \quad E_{n+1} = \cosh(E_n)^2. \quad (1)$$

The corresponding Wolfram construction is

```
Nest[Cosh[#]^2 &, a, n]
```

Its native leaf count is $1 + 3n$: each stage adds the heads `Cosh` and `Power`, and the exponent 2. At $n = 12$ this is only 37 leaves. The correct conclusion of the restricted proof procedure with empty facts is inconclusive. This witness does not assert that the expression is nonreal; it demonstrates excessive work before returning the same lack of proof.

The direct helper query is

```
AsymptoticAnalysis`Mathics`mathicsRealProof[
  Nest[Cosh[#]^2 &, a, n], {{}}, {{}}, 24]
```

A prospective public Mathics stress case is a package-only expansion with such a fixed coefficient, for example $E_n + x$, without a realness assumption on a . This is a *reachability and timing specification*, not an observed public Mathics result. Earlier normalization and Mathics-specific evaluation may change the precise public call count. With $a \in \mathbb{R}$, the same family supplies successful realness controls and must not be penalized or misclassified by the repair.

4.3 An exact call-count recurrence

Count entries into the bodies of the two proof helpers, but not the fact lookup, numeric predicates, expression evaluation, or other primitive work within each body. Write $R_n(d)$ for the total count of a realness query for E_n with depth d and empty facts.

Proposition 1 (Restricted-family recurrence). *For the source branches exercised by (1),*

$$R_0(d) = 1, \quad R_n(d) = 1 \quad (d \leq 0), \quad (2)$$

$$R_n(1) = 2, \quad R_n(2) = 7 \quad (n \geq 1), \quad (3)$$

$$R_n(d) = 5 + 2R_{n-1}(d-2) + R_{n-1}(d-3) \quad (n \geq 1, d \geq 3). \quad (4)$$

Proof. The symbolic base case has no applicable realness rule and makes no recursive real/sign call. A nonpositive depth returns immediately after the counted entry. At depth one, the power's realness body and its exhausted exponent-realness query contribute two entries. At depth two, the exponent is established, the sign query for the cosine makes three entries, and the later realness query for that cosine makes two, for a total of seven.

For $d \geq 3$, the power's realness entry and the realness check of its numeric exponent contribute two. Its sign query for $\cosh(E_{n-1})$ contributes its own entry, a query for E_{n-1} at depth $d-2$, and the realness fallback on that cosine. That fallback contributes one entry and a query for E_{n-1} at depth $d-3$. Finally, the integer-power branch independently queries the realness of the cosine: one entry plus another query for E_{n-1} at depth $d-2$. Summing yields (4). Every relevant realness subquery remains inconclusive for the empty-fact symbolic family, so none of these branches is skipped by a successful proof. $\square$

Corollary 1 (Initial growth). *Whenever $d \geq 3n$,*

$$R_n(d) = \frac{7 \cdot 3^n - 5}{2}. \quad (5)$$

Proof. At that depth, the recurrence never encounters a truncated power query. Its three recursive subqueries therefore have the same untruncated count. Starting with $R_0 = 1$, solve $R_n = 3R_{n-1} + 5$. $\square$

This is not a claim of exponential growth for arbitrarily large n while d stays fixed. For fixed d , the family saturates once $n \geq \lceil d/2 \rceil$. Let $C(d)$ denote that plateau. It obeys

$$C(0) = 1, \quad C(1) = 2, \quad C(2) = 7, \quad C(d) = 5 + 2C(d-2) + C(d-3).$$

Writing $F_0 = 0, F_1 = 1$ for the Fibonacci numbers gives the exact expression

$$C(d) = 4F_d + 2F_{d+1} + \frac{3}{2}(-1)^d - \frac{5}{2}. \quad (6)$$

It follows either by substitution and the three initial values, or from the homogeneous characteristic polynomial $(\lambda + 1)(\lambda^2 - \lambda - 1)$. Thus the depth-limited plateau itself grows exponentially with the permitted depth. In particular, $C(24) = 335,521$.

4.4 Measured and modeled results

The native-host execution of the retrieved, instrumented helpers returned

$$1, 8, 29, 92, 281, 848, 2549$$

for $n = 0, \dots, 6$, always with result **None**. These are actual helper-body counts, not timings. The larger table below comes from the independent Python model and is checked against (4).

n	Leaves	Original body entries	Memoized body entries	Count ratio
0	1	1	1	1.0
2	7	29	17	1.7
4	13	281	48	5.9
6	19	2,549	91	28.0
8	25	22,961	146	157.3
9	28	68,673	170	404.0
10	31	184,481	186	991.8
11	34	311,969	196	1,591.7
12	37	335,521	200	1,677.6
14	43	335,521	200	1,677.6

The rightmost column is *not a runtime speedup*. Expression-key construction, fact processing, language dispatch, and cache maintenance have costs not counted here. The model does not emulate `Mathics`, and it does not establish a memory or latency figure for a public expansion. It establishes a large and avoidable duplication factor in the inspected proof graph. The native smaller cases independently validate the relevant branch accounting.

4.5 A repair with a precise equivalence obligation

The appropriate local repair is to evaluate each pure proof subquery at most once per request. Its identity must include the operation (realness or signs), the normalized expression, the retained facts or an immutable identity for them, and the remaining depth. An inconclusive answer is cacheable for that exact key. It must not be confused with a proved falsehood.

Depth is essential. A shallow query about a known-real expression can return `None` while the deeper query proves it real. Facts are essential for the analogous distinction between unassigned a and $a \in \mathbb{R}$. A global memo table keyed only by expression would change the reasoning contract and risk stale assumptions.

For a pure, stable expression/fact snapshot, correctness follows by induction on remaining depth. A first visit evaluates the original body and stores its answer. Every recursive dependency has smaller depth, so already stored dependencies agree with the original procedure by induction. A repeated visit returns the identical stored answer. This proves preservation of the procedure's answers, not completeness of its mathematical reasoning.

A request-local memoized graph has at most one body evaluation per reachable key. With $O(n)$ distinct subexpressions and depth budget D , the number of candidate keys is $O(nD)$ for this family and one fixed fact set. That bound concerns graph nodes; it does not assume constant-time hashing or matching of large symbolic expressions. The supplied `proof_cost_model.py` demonstrates the scheduling change on an immutable restricted grammar and tests assumption isolation, depth-sensitive keys, successful realness controls, and explicit model-budget exhaustion.

The model is intentionally not offered as a complete replacement for `MathicsAssumptions.wl`. Production integration must decide how to scope the memo table, how to preserve existing evaluation semantics for user-defined or effectful expressions, and how interrupted requests release local state. Sharing is immediately justified for the pure built-in fragment of the witness; broader input classes require their own preservation tests. No global System definitions should be changed for this repair.

4.6 Focused acceptance criteria

The first production check should execute the direct helper family in both supported interpreters and both package layouts, recording answers and helper-body counts separately from elapsed time. The existing small-depth sequence supplies an exact baseline. A memoized implementation should preserve all answers while making repeated visits cache hits rather than full body evaluations.

Successful controls should include retained realness facts, positive bases, and different depth budgets in the same request. A fresh request with changed assumptions must not reuse the old answer. An interrupted computation must not retain a half-populated cross-request cache. These are specific acceptance obligations of the proposed fix, not a new general caching architecture.

Finally, run the proposed coefficient-containing public expression in Mathics with and without a realness assumption. Record which public path reaches the helper, what preprocessing changes the expression, and whether the eventual result is an expansion, a justified refusal, or a timeout. That end-to-end characterization remains outstanding; a smaller helper count alone must not be advertised as a package-wide acceleration.

5 N02: documentation checks disappear under optimization

5.1 Five assertions implement production validity gates

The complete retrieved `validation/check_documentation.py` contains five `assert` statements. Four check label uniqueness, resolution of mathematical references, presence of bibliography entries, and separation of software content from the mathematical article. The fifth checks archive equality inside a loop over six preserved engineering files. Afterward, the function unconditionally reports `"MissingReferences":0` and `"MissingCitations":0`.^[4]

Python's `-O` mode removes assertion statements; `-OO` also removes docstrings. The environment variable `PYTHONOPTIMIZE` can select corresponding optimization behavior. These are interpreter semantics, not a speculative optimization by a downstream compiler.^[10, 11]

Consequently, an optimized invocation can skip precisely the predicates that would justify those clean report fields. Other checks remain active: the guide builder, link checker, encoding checker, reads, and Git subprocess calls are not removed merely because this module uses assertions. The finding is a selective gate bypass, not a claim that every possible error becomes invisible.

5.2 Controlled execution of the actual checker

The bundle contains a verbatim copy of the retrieved upstream file. Its reconstructed Git blob identity was checked against the connector's returned blob identity during preparation; it is not a rewritten model of the checker. Each fixture supplies a small mathematical document and six archive files, while unrelated dependency providers return explicit fixture records.

The following results summarize the decisive cases:

Deliberate defect	Original Python	Original -O	Candidate, both modes
Duplicate active labels	Reject	Accept	Reject
Reference without a label	Reject	Accept	Reject
Citation without bibliography entry	Reject	Accept	Reject
Forbidden software content	Reject	Accept	Reject
Changed archived PDF bytes	Reject	Accept	Reject

For the missing-reference fixture, optimized execution returns a success object containing zero labels, one mathematical reference, and `"MissingReferences":0`. The process containing the checker then exits zero. For archive corruption, the historical and current fixture bytes are actually different; optimized execution nevertheless reaches the success object.

These observations do not allege that a published historical receipt was created under optimization or that the current article has a broken reference. They demonstrate that the current validity predicate depends on a process-startup mode that the function does not reject or accommodate. The ordinary negative controls verify that the same fixtures are meaningful in nonoptimized execution.

5.3 The candidate fix

A validation condition should use an ordinary branch and explicit exception, not a removable assertion statement. The small supplied helper is:

```
def require(condition: bool, message: str) -> None:
    if not condition:
        raise AssertionError(message)
```

Raising `AssertionError` explicitly preserves the existing ordinary failure class; optimization does not remove an explicit `raise`. This is intentionally different from replacing the predicate with another `assert`. The emitted candidate changes all five statements to `require(...)` calls and leaves the successful receipt schema and the archive CRLF normalization policy unchanged.

The patch generator refuses unexpected source anchors or an unexpected assertion count and never overwrites its input file. It is targeted to this pinned file shape, not a general source transformation tool. The supplied unified patch also adds the helper module to `validation`, so the new import has a concrete destination.

A narrower alternative is to refuse optimized execution at startup. That would prevent false success, but would make an otherwise valid validation command unavailable in that mode. Explicit checks avoid the needless restriction and make the function correct when imported as well as when run as a script.

5.4 Acceptance and remaining scope

Normal and optimized execution should agree on the acceptance decision for each tested fixture. The real guide/link/encoding/Git integrations must then be rerun in the repository, because deterministic substitutes cannot validate those dependencies. The unchanged success path and accepted CRLF-normalized archive control are important: a fix that simply refuses every run would satisfy the negative witnesses but be useless.

The bundle separately verifies the explicit `require` helper under normal, `-O`, and `-OO` execution. The full 56-observation checker matrix uses ordinary and `-O` execution; it is not mislabeled as a three-mode

full matrix. No recommendation is made to eliminate all debugging assertions from unrelated code. The repair applies to these production acceptance predicates.

6 N03: comments alter mathematical cross-reference validity

6.1 Source text is not the same as active TeX

The checker discovers immediate `\input` files and then extracts `\label`, `\ref`, `\eqref`, `\pageref`, `\bibitem`, and `\cite` using regular expressions over unfiltered source text. It does not distinguish an ordinary TeX line comment from an active command.^[4]

This defect is independent of N02. It occurs with assertions fully enabled, and merely replacing `assert` statements leaves it present.

A minimal masking witness is:

```
See \ref{ghost}.
% \label{ghost}
```

The original checker counts one label and one reference, sees no missing key, and returns a clean result. Real LaTeX ignores the commented-out definition and reports `Reference ‘ghost’...undefined`. Replacing the reference with `\cite{ghost}` and the comment with a commented-out `\bibitem{ghost}` gives the analogous false acceptance for a citation.

The opposite problem also occurs. A valid active label followed by a commented-out copy is counted twice, causing a false duplicate-label rejection. A commented `\input{does-not-exist}` is treated as a dependency and produces `FileNotFoundError`, although the compiler never attempts to load it.

6.2 Compiler cross-checks

Five actual small documents were compiled twice with `pdflatex`. The second-pass logs distinguish the cases without relying on a stale first-pass cross-reference state:

Document	Original checker	Actual two-pass compiler
Valid active reference	Accept	No reference warning
Active reference, commented definition	Accept	Undefined reference warning
Active citation, commented entry	Accept	Undefined citation warning
One active label, commented duplicate	Reject	No duplicate/reference warning
Commented nonexistent input	Reject	Compiles without loading that file

All ten compiler invocations exit zero, including the deliberately undefined-reference cases. Thus checking the compiler’s process status alone would not independently validate the checker’s zero-missing-reference claim. The relevant evidence is the compiler’s interpretation of active definitions and its warning output, preserved in `evidence/tex-oracles.json` and the accompanying logs.

These are synthetic documents. The review does not claim that the pinned mathematical article currently contains one of these mistakes. The demonstrated defect is in how the validation tool decides and reports correctness.

6.3 A narrow lexical repair

The candidate masks ordinary percent comments before discovering inputs and extracting mathematical keys. It preserves line boundaries and keeps original source bytes available for encoding, archive, and separation checks. Percent signs preceded by an odd run of backslashes are treated as escaped; an even run permits a comment to begin under standard TeX catcodes.

Preserving token boundaries is important. Blindly deleting a comment and its newline can accidentally concatenate fragments in a character-level scanner. The supplied masker replaces comment characters by spaces while retaining line endings. A test ensures that a fragment such as `\la`, a comment, and later `bel` does not become a newly invented `\label` token.

The extraction changes are deliberately local. Immediate input discovery uses the masked master. After the existing one-level concatenation, label/reference/citation extraction uses a masked copy. Archive comparisons still use unmodified bytes. The existing software-separation policy still examines raw text, so this change does not silently decide that software material in comments is permitted.

The implementation is not a complete TeX parser. It does not implement arbitrary catcode changes, `\verb`, verbatim environments, macro-generated labels, or unrestricted recursive input expansion. Such features require a declared parsing policy or a compiler-derived check. The ordinary comment witnesses are already sufficient to establish the bug; claiming a universal parser would exceed both the patch and the evidence.

6.4 A useful next development step

The focused invariant is that inserting or deleting genuinely inactive ordinary comments must not change mathematical cross-reference validity. The new fixtures test that invariant in both directions: comments must neither manufacture a definition nor create a duplicate or dependency.

After the narrow repair, the maintainers can define the admitted source grammar explicitly and decide how unsupported TeX constructs are handled. For syntax beyond that grammar, comparison against cross-reference information produced by the same actual compiler run is more reliable than treating every command-shaped substring as active source. Such reconciliation should identify unresolved keys and compiler warnings, not merely rely on a zero process exit status.

This is an extension of the mathematical-reference validity check, not another proposal about build-recorder provenance or publication transactions. It should be accepted separately from N02: an optimization-invariant but comment-blind checker is still wrong, and a comment-aware checker whose validity predicates disappear is still wrong.

7 Implementation package and reproducibility

7.1 What the supplied code does

Artifact	Purpose and status
<code>code/proof_cost_model.py</code>	Executed restricted proof model, exact independent recurrence, memoizing reference schedule, and bounded model execution. Not a complete CAS implementation.
<code>code/documentation_audit_guard.py</code>	Executed explicit acceptance predicate and standard-comment masker. Not a general TeX parser.
<code>code/check_documentation_candidate.py</code>	Generated candidate using the unchanged repository dependency interfaces. Exercised with fixtures, not integrated into the live repository.
<code>code/documentation-candidate.patch</code>	Unified candidate patch for the checker and added helper module. The repository was not modified.
<code>code/patch_documentation.py</code>	Reproducible guarded generator that writes a new output and refuses changed anchor/count shapes.
<code>code/cas_controls.wl</code>	Local-path specification for the seven finite-expression controls; native connector executions are recorded separately. Mathics execution remains unrun.
<code>code/cas_proof_counter.wl</code>	Fresh-process helper instrumentation with restoration of the original helper definitions. Mathics execution remains unrun.
<code>tests/</code> and <code>evidence/</code>	Runnable fixtures, model and guard tests, raw observations, compiler logs, environment record, and native transcriptions.

The original checker is preserved in `upstream/check_documentation.py` for comparison. It is intentionally defective on the counterexamples: a failing negative fixture is not a failure of the review's reproducer. The candidate and the upstream copy must not be confused when integrating changes.

7.2 Reproducing the local evidence

From the extracted archive's top-level directory:

```
python -S tests/run_evidence.py
python -S -m unittest discover -s tests -p 'test_*.py' -v
python -S code/proof_cost_model.py
```

The `-S` option only avoids unnecessary site initialization for these standard-library-only fixtures. It does not disable assertions. The evidence runner explicitly starts both normal and `-O` child interpreters for the optimization comparison. The LaTeX experiments require an installed `pdflatex`; recorded outputs are included for environments without it.

A direct counterexample can be reproduced with:

```
python -S tests/documentation_fixture.py \
    upstream/check_documentation.py comment_mask_reference
python -S -O tests/documentation_fixture.py \
    upstream/check_documentation.py missing_reference
python -S tests/documentation_fixture.py \
    code/check_documentation_candidate.py comment_mask_reference
```

The first two commands accept their deliberately invalid fixtures; the third rejects its fixture and exits nonzero. That nonzero exit is the expected candidate behavior.

For a fresh CAS process, set `$AuditRepositoryRoot` to a checkout of the pinned commit and load the appropriate script. The proof counter defaults to $n \leq 6$, the range actually observed on the native host; larger values are opt-in. Its temporary instrumentation is not intended to run concurrently with unrelated work in the same kernel.

7.3 Testing the candidate in a real checkout

The documentation candidate can be applied to an isolated checkout with the supplied unified patch. Review the diff first, retain the original fixture controls, and then invoke the repository’s real documentation checker with its actual guide builder, link checker, encoding checker, Pandoc, and historical Git objects. The fake providers in this bundle are a deliberate isolation mechanism and must not be installed in the repository.

The proof-search change is at a different maturity level. Its scheduling repair is demonstrated by a restricted immutable model and a preservation proof. It still needs an actual Mathics implementation and a native/Mathics characterization of the same pure fragment, including interruption and assumption-scope controls. No automatic patch to the entire proof engine is supplied or represented as tested.

8 Prioritized, finding-specific development plan

The two documentation fixes are small enough to review together but should retain separate tests and disposition records. First make acceptance predicates optimization-invariant. Then make ordinary comments invisible to mathematical-key extraction and dependency discovery. Preserve positive controls and raw-source policies. This closes two independent ways that a clean-looking receipt can be unsupported by its actual predicates.

For N01, first reproduce the helper counts in Mathics rather than guessing at an elapsed-time improvement. Then implement request-local sharing for a pure admitted fragment, using keys that retain operation, expression, facts, and depth. Check equivalent answers before measuring runtime. Only then broaden the sharing scope, after examining evaluation effects and backend expression canonicalization.

The final acceptance record should be specific: which checker modes, which comment grammar, which helper family, which runtime, and which public path were actually exercised. The existing review registers already cover wider architectural and coverage goals. They need not be reintroduced as newly discovered recommendations in order to integrate these three deltas.

9 Conclusion

This review’s strongest new result is an explicit account of how a small nested coefficient can cause extensive repeated work in the Mathics proof program. The exact recurrence, the native-host source-helper observations, and the independently checked memoizing model agree on the relevant mechanism. The default depth cap makes the recursion finite, but does not make its duplicated work small.

The documentation findings are more directly executable: the actual checker accepts constructed invalid inputs when assertions are optimized away, and it accepts undefined TeX references when their only apparent definitions are comments. The supplied candidate addresses those two mechanisms and passes

the focused bundle tests, with compiler controls confirming the lexical distinction.

No new incorrect asymptotic formula is claimed. Seven selected native package controls succeeded; broad mathematical and cross-interpreter correctness remains a separate qualification task. The practical output of this audit is three scoped findings with explicit evidence, two runnable tooling repairs, a proof-search scheduling prototype, and reproducible acceptance criteria rather than another restatement of the existing backlog.

A Evidence inventory

File	Interpretation
<code>evidence/python-tests.txt</code>	Nineteen passing bundle test methods; includes model tests, candidate checks, live checker probes, and receipt consistency checks. Not MUnit results.
<code>evidence/documentation-fixtures.js</code> <code>on</code>	56 observations: 14 fixtures, two checker versions, two Python modes. Expected rejections and false acceptances are preserved.
<code>evidence/proof-costs.json</code>	Fifteen modeled nesting levels of the witness family, original and memoized counts, and independent recurrence counts.
<code>evidence/tex-oracles.json</code>	Five real two-pass compiler experiments; undefined-reference/citation warnings are expected in two cases.
<code>evidence/tex-*.log</code>	The actual final-pass logs for those controlled compiler experiments.
<code>evidence/wolfram-observations.md</code>	Manually transcribed connector inputs and results for seven package controls and the retrieved proof helpers.
<code>evidence/environment.json</code>	Local Python/platform information and explicit boundaries between native, model, fixture, and unexecuted Mathics work.

References

- [1] Vladimir Reshetnikov, *Asymptotic*, reviewed commit 8cee870994f506b501bae3ea6bd4a3a7edb895c1. <https://github.com/VladimirReshetnikov/Asymptotic/commit/8cee870994f506b501bae3ea6bd4a3a7edb895c1>. Accessed 10 September 2026.
- [2] *MathicsAssumptions.wl*, especially `mathicsRealProof`, `mathicsSignProof`, and `mathicsAssumptionSimplify`, pinned repository source.
- [3] *MathicsSimplification.wl*, pinned repository source.
- [4] *check_documentation.py*, especially lines 27–39, 47–53, and 54–61. The full retrieved file is included in the bundle.
- [5] *Code-review status and complete finding crosswalk*, pinned repository record.
- [6] *Wave 3: findings, proposals, and implementation scope*, pinned repository record.
- [7] *Wave 4: findings, proposals, and implementation scope*, including its full crosswalk and proposal groups, pinned repository record.
- [8] *Wave-5 code-review index*, pinned repository record.
- [9] *Asymptotic: Incremental Repository Audit*, report 44, retained TeX source, especially its executive finding list and additional proposal boundaries.

- [10] Python Software Foundation, *Command line and environment*, options `-O`, `-OO`, and `PYTHONOPTIMIZE`. <https://docs.python.org/3/using/cmdline.html#cmdoption-O>. Accessed 10 September 2026.
- [11] Python Software Foundation, *The assert statement*. https://docs.python.org/3/reference/simple_stmts.html#the-assert-statement. Accessed 10 September 2026.